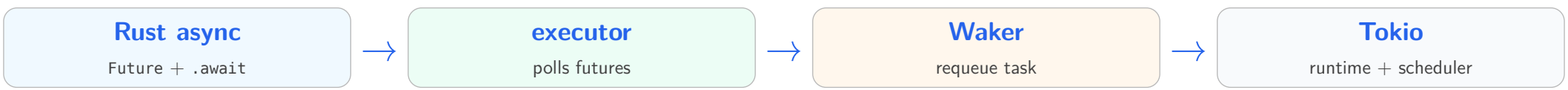
Introduction to async and Tokio

`async fn` creates a `Future`; Tokio polls it, parks it when it returns `Pending`, and wakes it when I/O or timers may progress.



Rust defines the async contract (`Future`, `.await`, `Waker`); Tokio supplies the runtime machinery: ready queues, worker threads, timers, and I/O events.

Problem 1: Blocking calls do not scale to many waits

Blocking APIs are simple until many calls wait

The traditional API story starts with blocking calls: code is easy to read because the stack itself remembers where execution should continue.

Blocking style: one flow, one stack

```c
int n = read(fd, buf, cap);
write(fd, buf, n);
```

While read waits, the OS thread is parked: it keeps a stack, occupies scheduler state, and must be woken later.

Potential solution: multiplex one worker

many fds connections → **event loop** poll events → **dispatch** only ready work

One worker can run whichever operation can make progress instead of blocking on one fd.

Blocking API

Sequential and readable; the call stack stores the continuation.

Many waiting threads

- Thread creation costs; stacks consume memory;
- wakeups and context switches go through the kernel scheduler.

Solution: Multiplexing

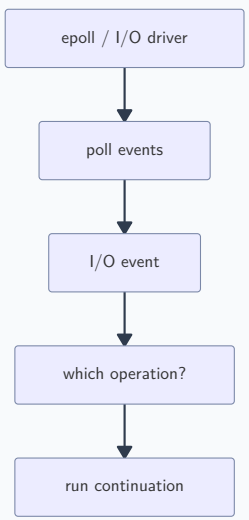
Use fewer workers for many waits; move each paused operation's continuation out of the OS thread stack.

Interrupt is slow, so RDMA/dpdk use polling instead of interrupts. Blocking is not even an option.

Multiplexing creates a continuation problem

Once one thread serves many operations, the stack can no longer be the only place that remembers every paused operation.

What the event loop knows



What each operation must keep

where was I?

next instruction / state

what did I keep?

locals, buffers, offsets

who calls me?

callback or waker

This is the suspension problem. C callbacks, Go goroutines, and Rust/C++ stackless coroutines are different ways to represent the paused continuation.

Problem 2: Paused work needs a continuation

Multiplexing frees the worker thread, but it also removes the thread stack as the place that remembers each operation's progress.

C callbacks make the continuation explicit

In manual event-loop code, the programmer moves the needed locals into a context struct, starts an async operation, and passes a callback as "the rest of the function."

Manual transformation

locals
fields in `Ctx`

next line
callback function

wake path
event loop calls callback

The callback is the continuation: it receives the context and starts the next async operation.

Read, then write, in callback style

c

```c
typedef struct { int fd; char *buf; } Ctx;

void start(Ctx *c) {
  read_async(c, on_read);   // returns now
}

void on_read(Ctx *c) {      // callback
  write_async(c, on_write);
}
```

Problem: Composability

Different async providers have different callback protocols. The programmer must hand-glue them together.

The logical operation is simple

race
`read_file` vs 100 ms timer

→

then parse
CPU pool

→

then report
event loop

One operation: sequencing + race + scheduler hops + errors + cleanup.

What the programmer must write

c

```c
LoadCtx *ctx = new LoadCtx(...); // owns buf
read_file(f, buf, n, &ctx->io, on_read);
start_timer(loop, 100, on_timeout, ctx);

on_read(...) {
  if (!claim_winner(ctx)) return;
  post_work(pool, parse, ctx);
}
on_timeout(...) {
  if (!claim_winner(ctx)) return;
  cancel_read(&ctx->io);
}
```

Why this hurts the programmer

invent state
`LoadCtx` owns everything

prove race
exactly one winner

late events
cancel may still call back

re-glue
repeat for each API mix

These are correctness chores, not the application logic. Miss one and you get use-after-free, double completion, leaked buffers, wrong-thread continuation, or lost errors. A uniform operation shape lets `timeout(read)` and `then(parse)` be reusable steps instead.

The glue is a hand-written protocol

To compose the callbacks, the programmer has to create the missing operation object. We specifically face many problems like this from the **SG-IOV** work.

1. Invent shared operation state

c

```c
typedef struct LoadCtx {
  overlapped io;
  timer_handle *timer;
  pool *cpu; loop *ev;
  char *buf; int len;

  atomic_bool done;
  atomic_int refs;
  void (*finish)(Result, void *);
  void *user;
} LoadCtx;
```

This struct exists only because the call stack can no longer hold the operation's continuation and lifetime.

2. Make every callback obey it

c

```c
bool claim(LoadCtx *c) {
  return !atomic_exchange(&c->done, true);
}

void on_read(int st, int n, overlapped *io) {
  LoadCtx *c = container_of(io, LoadCtx, io);
  if (!claim(c)) { release(c); return; }
  cancel_timer(c->timer);
  if (st) finish_error(c, st);
  else post_work(c->cpu, parse, c);
}

void on_timeout(void *p) {
  LoadCtx *c = p;
  if (!claim(c)) { release(c); return; }
  cancel_read(&c->io); // callback may still arrive
  finish_error(c, ETIMEDOUT);
}
```

Solution: Compiler to separate async logic from continuation mechanics

The win is local control flow: read with timeout, then parse, then report. The API carries the continuation machinery.

Go: goroutine + select

```go
go func() {
  select {
  case r := <-readDone:
    parseOnPool(r)
  case <-time.After(100*ms):
    cancelRead()
  }
}()
```

One goroutine body; `select` names the race.

Rust/Tokio: Future + .await

```rust
let n = timeout(dur,
read()).await??;
let x = spawn_blocking(||
parse(n)).await??;
report(x).await?;
```

One async block; `.await` marks where it may pause.

C++: coroutine or sender

```cpp
timeout(async_read(), 100ms)
  | let_value(parse_on(cpu))
  | continues_on(loop)
  | then(report);
```

One pipeline; adapters name the next step.

Go is stackful; Rust/C++ futures are stackless

Go preserves a suspended call stack, like a lightweight userspace thread. Rust and C++ async/coroutine styles compile suspension points into state-machine objects.

Go: stackful goroutine

goroutine stack

call frames + locals + next instruction

1. More like a userspace thread: it has its own stack.
2. Suspension keeps the call stack intact across function calls.
3. The Go runtime parks, grows, moves, and resumes stacks.

Rust/C++: stackless future/coroutine

future value

enum-like state machine + live locals

1. Suspension is explicit: Rust `.await`; C++ `co_await`, `co_return`, `co_yield`.
2. The compiler builds an object/state machine instead of requiring a runtime stack.
3. Zero-cost principle: pay mainly for the state you save and the scheduling you use.

The shared idea is stackless suspension: the compiler rewrites a sequential-looking body into an object that can stop and resume at explicit suspension points.

Rust (and also C++ coroutines) stores the continuation in a `Future`

Rust's language-level answer is a stackless `Future` (let the compiler to write the dirty codes): `async fn` creates a value that stores saved locals and nothing makes progress until some executor polls it.

1. Language syntax

```rust
async fn fetch() -> Bytes {
  read().await
}
```

`async fn` does not choose an event loop. It returns a value implementing `Future`.

→

2. Future trait

`poll`

Ready or Pending

A future is a state machine plus a standard `poll` interface.

→

3. Executor needed

runtime

Tokio, async-std, custom

The executor decides when and where to poll futures.

Keep the boundary clear: Rust provides `Future`, `.await`, and `Poll`; Tokio connects them to timers, sockets, worker threads, and scheduling.

The sequential illusion

`async fn` looks sequential, but calling it creates a future; `.await` is where that future may give control back to Tokio.

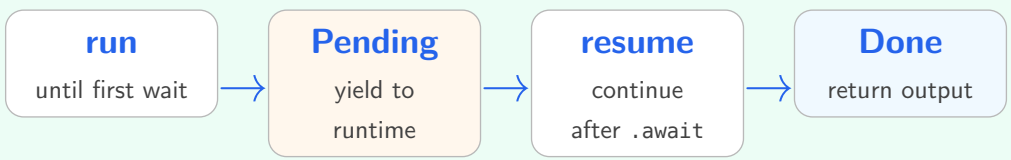
What we write

rust

```rust
async fn handle() {
  let a = read().await;
  let b = compute(a);
  write(b).await;
}
```

Calling `handle()` alone does not run all of this immediately; it produces a future that must be awaited or spawned.

What happens over time



When the future returns `Pending`, the thread can run another task instead of blocking.

One future stores state; `.await` defines transitions

A future is a value that stores "which await am I blocked on?" plus the child future and locals needed to resume.

Source: only `.await` is the suspension marker

rust

```rust
async fn handle() {
  let req = read_req().await;
  let resp = route(req).await;
  write_resp(resp).await;
}

tokio::spawn(handle());
```

> `spawn`: hand the future to Tokio

spawn does not run the body inline; it registers the future as a Tokio task.

Pseudo-code: compiler-built future

rust

```rust
// state = Start | Read(f) | Route(f) | Write(f)
fn poll(&mut self, cx: &mut Cx) -> Poll<()> {
  loop { match &mut self.state {
    Start => self.state = Read(read_req()),
    Read(f) => match f.poll(cx) {
      Pending => return Pending, // keep Read
      Ready(req) => self.state = Route(route(req)),
    },
    Route(f) => match f.poll(cx) {
      Pending => return Pending, // keep Route
      Ready(resp) => self.state = Write(write_resp(resp)),
    },
    Write(f) => return f.poll(cx),
  }}
}
```

The future owns the current state and live locals. `.await` is the boundary where polling may yield `Pending` and resume from the saved state later.

Problem 4: Futures do not run themselves

A `Future` is inert until a runtime polls it; Tokio supplies the workers, event loop, ready queue, and wakers.

Tokio adds scheduling and I/O events

Tokio connects Rust futures to a concrete executor, task scheduler, and event sources such as timers or I/O.

1. Your async code

Future

stores progress and local variables

Returns `Ready` when done, or `Pending` when it must wait.

→

2. Tokio executor

Task scheduler

polls ready futures

It does not spin forever; it waits for a wakeup signal.

→

3. I/O or timer event source

Waker

requeue this task later

Timers, sockets, or other events call wake when progress may be possible.

Future = saved computation; Tokio executor = scheduler; Waker = callback path back into the scheduler.

Event loop code and Tokio task code share the same idea

Traditional network code makes the event loop explicit; Tokio hides that loop behind async tasks while keeping the same I/O event idea underneath.

Traditional event loop

rust

```rust
loop {
  evs = epoll_wait(epfd);
  for ev in evs {
    if ev.readable { rd(fd); }
    if ev.writable { wr(fd); }
  }
}
```

The programmer explicitly asks the OS which file descriptors are ready, then dispatches handlers.

Tokio task style

rust

```rust
tokio::spawn(async move {
  let n = sock.read(&mut buf).await?;
  sock.write_all(&buf[..n]).await?;
  Ok::<_, io::Error>(())
});
```

The task reads as sequential code. The runtime owns the event loop and wakes the task when the socket may progress.

Read `.await` as: try the operation now; if it would block, save this task's continuation and let Tokio run something else.

`Pending` saves a `Waker` for later

When a future cannot finish now, it returns `Pending` and records a `Waker` so something else can ask Tokio to poll it later.

Conceptual future implementation

rust

```rust
fn poll(self: Pin<&mut Self>) {
  if operation_is_ready() {
    Poll::Ready(result)
  } else {
    save(waker);
    Poll::Pending
  }
}
```

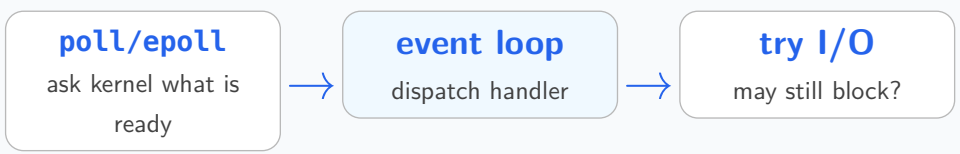
What the saved waker does

The waker is not the result. It is a handle for saying: “this task may now be worth polling again.”

I/O events find work; wakers requeue tasks

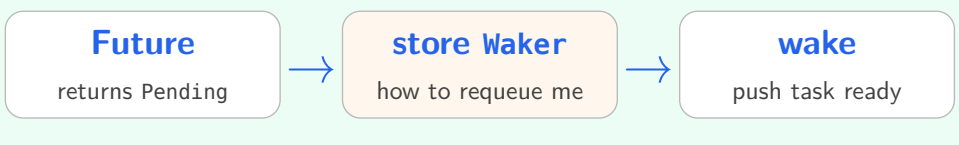
Tokio combines two familiar ideas from network programming: event polling asks "which socket or timer changed?", while wakers make task notification explicit.

1. Polling / reactive style



Traditional server code often loops over I/O events, then calls the operation that should now make progress.

2. Waker / proactive style



Tokio still waits for I/O and timer events under the hood, but each pending future leaves behind a precise callback path to the task that should be polled again.

`epoll` tells the runtime that a socket is ready; the `Waker` tells the runtime which suspended Rust task should be put back on the ready queue.

Poll, Waker, and Tokio executor

Tokio's `poll` is the "try to make progress" step; `Waker` is the requeue handle registered when progress is not possible yet.

Problem 5: Runtime-owned tasks need type guarantees

A spawned future can outlive the stack frame that created it, so Rust must prove what its saved state owns, borrows, and can move.

Borrow vs async move: view vs captured value

async move is like a move closure: variables used inside are captured by value into the future object. Non-Copy values move; Copy values are copied.

Borrow: future depends on caller

rust

```rust
let cfg = Cfg::load();

let fut = async {
  read_with(&cfg).await;
};
```

fut may use &cfg, so cfg must stay alive until fut is done.

async move: future captures by value

rust

```rust
let cfg = Cfg::load();

let fut = async move {
  read_with(&cfg).await;
};
```

cfg becomes a field inside fut. The &cfg inside the block borrows that stored field, not the caller's stack slot.

Lifetimes: how long a borrow may be used

Rust lifetimes answer one concrete question: can this reference still point to live data at every place it is used?

A reference is only a view

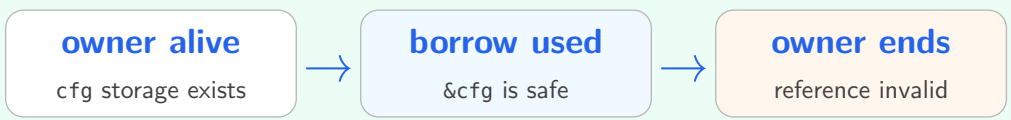
```rust
let cfg = Cfg::load();   // owner

{
  let r = &cfg;          // borrow
  use_cfg(r);
}                       // borrow ends

drop(cfg);              // owner ends
```

`&Cfg` does not own a `Cfg`; it is valid only while the owner is still alive.

What the compiler checks



1. A lifetime is a proof window, not a runtime timer.
2. An annotation names a relationship; it does not make data live longer.

Async challenge: `.await` can store a borrow for later

An async function with a borrowed argument creates a future whose saved state may contain that borrow across `.await`.

What the future may contain

rust

```rust
async fn read_cfg(cfg: &Cfg) {
  read_with(cfg).await;
}

// conceptual shape
struct ReadCfg<'a> {
  cfg: &'a Cfg,
  state: State,
}
```

When is that okay?

rust

```rust
// OK: future completes here
read_cfg(&cfg).await;

// Problem: task may outlive this stack
tokio::spawn(read_cfg(&cfg));

// Solution: give the task owned state
tokio::spawn(async move {
  read_cfg(&owned_cfg).await;
});
```

For shared config, move an `Arc<Cfg>` into the task.

Tokio tasks and `spawn`

`spawn` crosses an ownership boundary: the task may continue after the function that created it has returned.

Before spawn: caller owns the stack

caller frame
`cfg`, request, locals

→

create future
captures values

Local variables can disappear when the caller returns.

After spawn: runtime owns the task

worker 1
task A

worker 2
task A later

worker 3
other tasks

`tokio::spawn` stores the future in the runtime. The spawned future and its output must be `Send + 'static` on the multi-thread runtime.

`async move` transfers ownership into the task

> `spawn` means the runtime owns the task, so captured values must be safe to keep after the caller returns.

rust

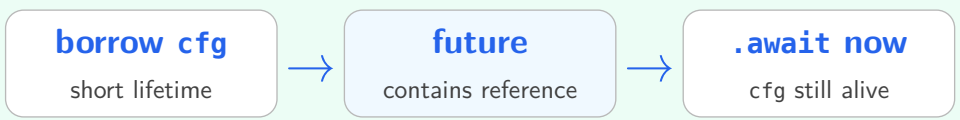
```rust
let shared = Arc::new(State::new());
let task_state = shared.clone();

tokio::spawn(async move {
    task_state.handle().await;
});
```

The lifetime challenge

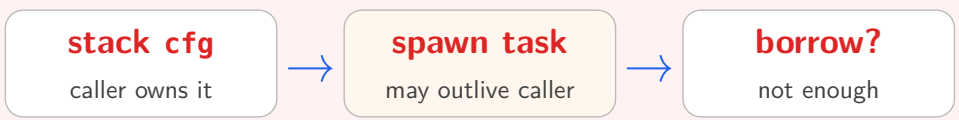
Lifetime errors prevent a saved future from containing references to stack data that may disappear before the future resumes.

OK: awaited in the same scope



A borrowed future can be fine when it is awaited before the borrowed value goes away.

Harder: spawned task



A spawned task usually needs owned data: move the value, clone it, or use `Arc`.

`static` in this context means "the future contains no short-lived borrowed references," not "the value leaks forever."

Local await bounds the borrow; spawn may let it escape

Borrowing across `.await` is okay only when the future cannot outlive the borrowed value.

Local await: borrow is bounded

rust

```rust
async fn use_cfg(cfg: &Cfg) {
    read_with(cfg).await;
}

use_cfg(&cfg).await;
```

Spawn: borrow may escape

rust

```rust
tokio::spawn(async {
    read_with(&cfg).await;
}); // not 'static

tokio::spawn(async move {
    read_with(&owned_cfg).await;
});
```

Send / Sync: safe to cross thread boundaries

Rust uses marker traits to decide what may cross threads: Send for moving ownership to another thread, Sync for sharing references between threads.

Rust's question

Send
can move T to another thread

→

Sync
can share &T with another thread

These are compile-time guarantees about the type. They do not add locks or make unsafe sharing safe at runtime.

Why Tokio requires them

poll
worker 1

→

.await
state saved

→

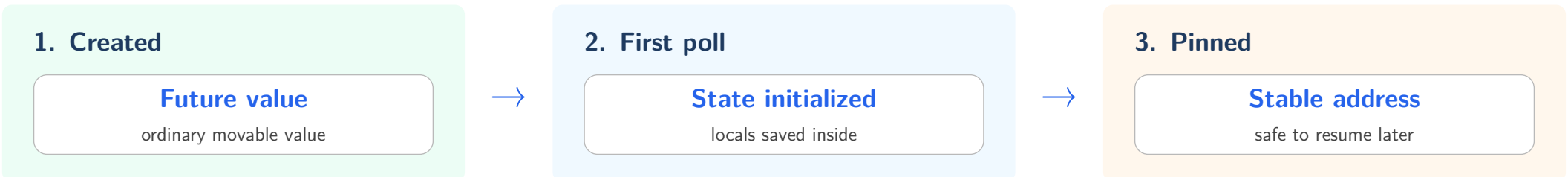
resume
worker 2

Multi-thread `tokio::spawn` may store a task, wake it later, and run it on another worker. Therefore the spawned future and output must be safe to move across threads.

If a task must keep `!Send` state, use single-thread-local execution such as `LocalSet` / `spawn_local` instead of multi-thread `tokio::spawn`.

`Pin` protects future state after polling starts

`Pin` says that after a future starts running, the memory its saved state may depend on must not be moved. This is address stability.



Many accelerators require submitted descriptors or buffers to stay at a stable address until completion. You cannot move the state after the device may use that address.

The `Future::poll` signature receives pinned state

> Application code usually awaits futures, but executor/library code polls a pinned future so saved state is not moved unexpectedly.

rust

```rust
trait Future {
    type Output;

    fn poll(
        self: Pin<&mut Self>,
        cx: &mut Context<'_>,
    ) -> Poll<Self::Output>;
}

let value = some_future.await;
```

Close: Reason from the saved continuation

The same question explains every later rule: what continuation is saved, who can wake it, and where may that saved state live or move?

Rules of thumb for Tokio code

Good Tokio code makes ownership and wakeup boundaries explicit.

Own when spawning

Use `async move`; clone or move owned values into tasks. Use `Arc` for shared long-lived state.

Keep borrows local

Borrow across `.await` only when the future is awaited within a scope that proves the borrow stays valid.

Watch cross-await state

Values used after `.await` become saved task state and may need `Send`.

Avoid locks across await

Do not hold blocking locks across `.await`; prefer scoped locks or async-aware synchronization.

Use channels

Communicate between tasks with channels instead of shared mutable state when possible.

Plan cancellation

Dropping a future or task can cancel work; make cleanup and ownership behavior intentional.

When code reaches `.await`, ask three questions: what state is saved, who wakes it, and can that saved state outlive or move with the task?